



Retail Sales SQL Project

A beginner-friendly SQL project that covers end-to-end data analysis on a retail sales dataset — from database setup and data cleaning to business-driven queries and insights.



Project Structure

retail_sales_project/

|

|— sql_retail_sales_project.sql # Main SQL file with all queries

|— README.md # Project documentation



Database & Table Setup

- **Database:** PROJECT1
- **Table:** retail_sales

Schema

Column	Data Type	Description
transactions_id	INT (PK)	Unique transaction identifier
sale_date	DATE	Date of sale
sale_time	TIME	Time of sale
customer_id	INT	Customer identifier
gender	VARCHAR(50)	Customer gender
age	INT	Customer age
category	VARCHAR(50)	Product category

Column	Data Type	Description
quantiy	INT	Quantity sold (<i>note: typo in original schema</i>)
price_per_unit	FLOAT	Price per unit
cogs	FLOAT	Cost of goods sold
total_sale	FLOAT	Total sale amount

Data Cleaning

Before analysis, the dataset was cleaned by:

1. **Checking for NULL values** across all columns
2. **Deleting the NULL row** using:

```
SET SQL_SAFE_UPDATES = 0;
```

```
DELETE FROM retail_sales WHERE transactions_id IS NULL;
```

```
COMMIT;
```

3. **Verifying** the dataset is clean — 0 NULL rows returned after cleaning
-

Data Exploration

```
-- Total number of sales
```

```
SELECT COUNT(*) AS total_sale FROM retail_sales;
```

```
-- Total distinct customers
```

```
SELECT COUNT(DISTINCT customer_id) FROM retail_sales;
```

```
-- Product categories available
```

```
SELECT DISTINCT category FROM retail_sales;
```



Business Questions & Queries

Q1. Sales on a specific date

```
SELECT * FROM retail_sales
```

```
WHERE sale_date = '2022-11-05';
```

Q2. Clothing sales with quantity ≥ 3 in Nov 2022

```
SELECT * FROM retail_sales
```

```
WHERE category = 'Clothing'
```

```
AND quantity >= 3
```

```
AND DATE_FORMAT(sale_date, '%Y-%m') = '2022-11';
```

Q3. Total sales per category

```
SELECT category, SUM(total_sale)
```

```
FROM retail_sales
```

```
GROUP BY category;
```

Q4. Average age of Beauty category customers

```
SELECT category, AVG(age)
```

```
FROM retail_sales
```

```
GROUP BY category
```

```
HAVING category = 'Beauty';
```

Q5. Transactions where total sale > 1000

```
SELECT * FROM retail_sales
```

```
WHERE total_sale >= 1000;
```

Q6. Transaction count by gender and category

```
SELECT category, gender, COUNT(*) AS total_transactions
```

```
FROM retail_sales
```

```
GROUP BY category, gender
```

```
ORDER BY category, gender;
```

Q7. Best selling month each year *(to be completed)*

-- Find average sale per month and best selling month per year

Q8. Top 5 customers by total sales

```
SELECT customer_id, SUM(total_sale) AS total_sales
```

```
FROM retail_sales
```

```
GROUP BY customer_id
```

```
ORDER BY total_sales DESC
```

```
LIMIT 5;
```

Q9. Unique customers per category

```
SELECT category, COUNT(DISTINCT customer_id) AS unique_customers
```

```
FROM retail_sales
```

```
GROUP BY category;
```

Q10. Orders by shift (Morning / Afternoon / Evening)

```
WITH hourly_sale AS (
```

```
    SELECT *,
```

```
    CASE
```

```
        WHEN EXTRACT(HOUR FROM sale_time) < 12 THEN 'Morning'
```

```
    WHEN EXTRACT(HOUR FROM sale_time) BETWEEN 12 AND 17 THEN 'Afternoon'

    ELSE 'Evening'

END AS shift

FROM retail_sales

)

SELECT shift, COUNT(*) AS total_orders

FROM hourly_sale

GROUP BY shift;
```

Tools Used

- **MySQL** — Database management
 - **MySQL Workbench** — Query execution and data visualization
 - **Excel** — Source dataset
-

Key Learnings

- Setting up a relational database from scratch
 - Cleaning dirty data using **IS NULL** checks and **DELETE**
 - Using aggregate functions: **SUM()**, **AVG()**, **COUNT()**
 - Filtering with **WHERE**, **HAVING**, and **DATE_FORMAT()**
 - Using **GROUP BY**, **ORDER BY**, and **LIMIT**
 - Writing CTEs (**WITH** clause) for complex queries
 - Understanding **CASE WHEN** for conditional logic
-

Known Issues

- Column **quantiy** has a typo (should be **quantity**) — kept as-is to match original schema
- Date format in source Excel is **DD-MM-YYYY** but MySQL stores as **YYYY-MM-DD**

 Author

Adnan Aspiring Data Analyst | SQL • Data Cleaning • Business Analysis